

UrbanHub

Documentation technique

Service d ingestion IoT et pipeline CI/CD

EC03 - Partie 2

Document anonyme - 29 juillet 2026

Java 25 • Spring Boot • Kafka • Docker • GitHub Actions • DevSecOps

Sommaire 1. Synthèse exécutive 2. Tutoriel de prise en main 3. Architecture 4. Référence API 5. Pipeline CI/CD
6. Guide d exploitation 7. Dépannage 8. Maintenance et évolution 9. Sécurité 10. Référence technique 11.
Journal des versions 12. Déclaration d utilisation de l intelligence artificielle 13. Conclusion

Objet et périmètre

Cette documentation décrit ingestion-service et la chaîne CI/CD qui protège sa livraison. Elle couvre la configuration, l'utilisation, l'API, l'architecture, l'exploitation, le dépannage, la sécurité, la maintenance et les évolutions prévues.

Résultats vérifiés

- Pipeline complet vert de install à deploy
- Tous les tests automatisés réussis
- Couverture des instructions ~78 % et des branches ~69 %
- Gates JaCoCo à 60 %
- Checkstyle bloquant et 0 finding SpotBugs SECURITY
- Gitleaks bloquant, Trivy et SBOM CycloneDX
- Image Docker Java 25 multi-stage et non-root
- Smoke tests readiness, HTTP 401 et HTTP 202 réussis

Synthèse exécutive

BLUF UrbanHub dispose désormais d'un service d'ingestion IoT industrialisé, testé et sécurisé pour un déploiement local reproductible. Le pipeline automatisé vérifie le code, les tests, la couverture, la qualité, les secrets, les dépendances et l'image Docker avant de lancer des smoke tests.

La plateforme réduit ainsi le risque d'accepter des données invalides, de publier un secret par erreur ou de livrer une version non testée. Les prochaines priorités sont la sécurisation complète de Kafka, l'identité distincte de chaque passerelle et la remédiation progressive des vulnérabilités de dépendances.

Valeur pour la ville

Fiabilité des données

Les mesures sont authentifiées, limitées et validées avant publication. Une donnée invalide ne doit produire aucun événement Kafka.

Réactivité opérationnelle

L'architecture événementielle permet de distribuer rapidement une mesure vers les services de qualité, d'analyse et d'alerte sans bloquer l'API d'entrée.

Réduction du risque

Le pipeline bloque :

- les tests en échec ;
- les violations de style ;
- une couverture insuffisante ;
- les secrets détectés ;
- une image Docker non construite ;
- un déploiement local non fonctionnel.

Maintenabilité La documentation Doc as Code, les contrats OpenAPI, les diagrammes et le changelog facilitent la reprise du projet et limitent la dette technique.

Résultats principaux

Indicateur	Résultat
Pipeline	6 jobs automatisés et enchaînés
Tests	Tous réussis

Indicateur	Résultat
Couverture des instructions Environ 78 %	
Couverture des branches Environ 69 %	
Finding SpotBugs SECURITY	0
Détection de secrets	Gate Gitleaks bloquante
Inventaire logiciel	SBOM CycloneDX
Déploiement	Docker Compose local validé

Smoke tests Readiness, HTTP 401 et HTTP 202 validés

Impacts métier

Impact financier

- réduction du temps consacré aux vérifications manuelles ;
- détection plus précoce des défauts, donc correction moins coûteuse ;
- environnement reproductible limitant les écarts entre postes ;
- documentation réduisant le coût de transfert de connaissance.

Impact écologique

- meilleure qualité des données utilisées pour suivre la pollution ;
- détection plus rapide des anomalies de mesure ;
- base technique pour déclencher des alertes et décisions urbaines ciblées ;
- image multi-stage limitant les composants inutiles déployés.

Impact opérationnel

- traçabilité par correlationId ;
- disponibilité contrôlée par healthcheck ;
- erreurs standardisées pour accélérer le diagnostic ;
- automatisation du build et des smoke tests.

Risques résiduels

Risque	Niveau	Recommandation
Clé API partagée distincte par passerelle	Modéré	Identité
Rate limiting en mémoire Gateway	Modéré	Redis ou API
Kafka local non chiffré Élevé avant production TLS, SASL et ACL		
Vulnérabilités de dépendances mises à jour	Modéré	Priorisation et

Métriques, traces et alertes Observabilité limitée Modéré centralisées

Recommandations prioritaires

Court terme

1. qualifier et corriger les vulnérabilités Trivy ;
2. terminer la qualification du finding SpotBugs ;
3. renforcer Kafka avec TLS, SASL et ACL ;
4. ajouter retry et DLQ.

Moyen terme

1. attribuer une identité à chaque passerelle ;
2. externaliser les quotas ;
3. centraliser logs, métriques et traces ;
4. automatiser les mises à jour de dépendances.

Long terme

1. déployer une API Gateway ;
2. signer les images ;
3. suivre la SBOM dans une plateforme dédiée ;
4. versionner le portail documentaire avec mike.

Conclusion La solution atteint un niveau opérationnel adapté à une démonstration industrialisée. Le pipeline, les tests, la sécurité applicative, la conteneurisation et la documentation fournissent une base solide. Le passage à la production nécessite principalement le renforcement des identités, du broker Kafka et de l'observabilité.

Tutoriel de prise en main

Objectif Ce tutoriel accompagne la première exécution de ingestion-service.

À la fin du parcours, vous aurez :

- vérifié le code avec Maven ;
- démarré Kafka et le service d'ingestion ;
- contrôlé l'état de santé du service ;
- envoyé une mesure IoT authentifiée ;
- vérifié la création d'un événement MeasurementReceived.

1. Vérifier les prérequis Depuis un terminal :

```
java --version
mvn --version
docker --version
docker compose version
```

Versions de référence :

Java 25
Maven 3.9 ou compatible
Docker avec Compose V2

2. Vérifier le service avec Maven Depuis la racine du dépôt :

```
mvn --file services/ingestion-service/pom.xml clean verify
```

La commande exécute notamment :

- Checkstyle ;
- les tests JUnit ;
- les tests REST et de sécurité ;
- le test non fonctionnel ;
- la génération du rapport JaCoCo ;
- les gates de couverture ;
- SpotBugs et Find Security Bugs ;
- le packaging du JAR.

Résultat attendu :

All coverage checks have been met.
BUILD SUCCESS

3. Définir une clé API locale

La route d'ingestion est protégée par le header X-API-Key.

Sous PowerShell :

```
$env:INGESTION_API_KEY = "urbanhub-local-tutorial-key"
```

Sous Bash :

```
export INGESTION_API_KEY="urbanhub-local-tutorial-key"
```

Cette valeur sert uniquement à l'environnement local. Elle ne doit pas être enregistrée dans Git.

4. Démarrer Kafka et ingestion-service Depuis la racine du dépôt :

```
docker compose up --build -d kafka ingestion-service
```

Afficher l'état des conteneurs :

```
docker compose ps
```

Les composants doivent être démarrés et le service d'ingestion doit devenir sain après son initialisation.

5. Vérifier la readiness Sous PowerShell :

```
Invoke-RestMethod `
    http://localhost:8082/actuator/health/readiness
```

Avec curl :

```
curl http://localhost:8082/actuator/health/readiness
```

Réponse attendue :

```
{
  "status": "UP"
}
```

6. Vérifier la protection de l'API Envoyer une requête sans clé API :

```
curl -i -X POST http://localhost:8082/api/ingestion/measurements -H "Content-Type: application/json" -d '{
  "zoneId": "ZFE-1",
  "stationId": "AIR-STATION-042",
  "indicator": "N02",
  "value": 220.5,
  "timestamp": "2026-07-27T08:00:00Z"
}'
```

Résultat attendu :

HTTP 401 Unauthorized

Cette vérification confirme que la route refuse une passerelle non authentifiée.

7. Envoyer une mesure valide Avec curl :

```
curl -i -X POST http://localhost:8082/api/ingestion/measurements -H "Content-Type: application/json" -H "X-API-Key: urbanhub-local-tutorial-key" -d '{
  "zoneId": "ZFE-1",
  "stationId": "AIR-STATION-042",
  "indicator": "N02",
  "value": 220.5,
  "timestamp": "2026-07-27T08:00:00Z"
}'
```

Sous PowerShell :

```
$body = @{
  zoneId = "ZFE-1"
  stationId = "AIR-STATION-042"
  indicator = "N02"
  value = 220.5
  timestamp = "2026-07-27T08:00:00Z"
} | ConvertTo-Json
```

```
Invoke-RestMethod `
  -Method Post `
  -Uri "http://localhost:8082/api/ingestion/measurements" `
  -Headers @{
    "X-API-Key" = $env:INGESTION_API_KEY
  } `
  -ContentType "application/json" `
  -Body $body
```

Résultat attendu :

HTTP 202 Accepted

Exemple de réponse :

```
{
  "status": "ACCEPTED",
  "correlationId": "identifiant-généré"
}
```

8. Comprendre l'événement publié

Après acceptation, le service publie un événement MeasurementReceived dans le topic Kafka measurements.received.

L'événement contient notamment :

```
{
  "eventId": "identifiant-généré",
  "eventType": "MeasurementReceived",
  "eventVersion": "1.0",
  "correlationId": "identifiant-de-corrélation",
  "occurredAt": "date-de-publication",
  "source": "ingestion-service",
  "zoneId": "ZFE-1",
```

```
"stationId": "AIR-STATION-042",
  "indicator": "N02",
  "value": 220.5,
  "timestamp": "2026-07-27T08:00:00Z"
}
```

correlationId permet de suivre la mesure dans les microservices situés en aval.

9. Tester une erreur de validation

Envoyer un indicateur inconnu et une valeur négative :

```
curl -i -X POST http://localhost:8082/api/ingestion/measurements -H "Content-Type: application/json" -H "X-API-Key: urbanhub-local-tutorial-key" -d '{
  "zoneId": "ZFE-1",
  "stationId": "AIR-STATION-042",
  "indicator": "UNKNOWN",
  "value": -1,
  "timestamp": "2026-07-27T08:00:00Z"
}'
```

Résultat attendu :

HTTP 400 Bad Request

Aucun événement Kafka ne doit être publié pour une mesure invalide.

10. Consulter Swagger Lorsque Swagger est activé :

<http://localhost:8082/swagger-ui.html>

Utiliser le bouton d'autorisation pour renseigner la clé API, puis exécuter la route d'ingestion depuis l'interface.

11. Consulter les logs docker compose logs -f ingestion-service

Pour Kafka :

```
docker compose logs -f kafka
```

12. Arrêter l'environnement docker compose down --volumes --remove-orphans

Résultat final

Le parcours est réussi lorsque :

- Maven affiche BUILD SUCCESS ;
 - la readiness retourne UP ;

- une requête sans clé retourne HTTP 401 ;
- une requête invalide retourne HTTP 400 ;
- une requête valide retourne HTTP 202 ;
- un événement MeasurementReceived est publié dans Kafka.

Architecture

Contexte UrbanHub est une plateforme événementielle de ville intelligente. Des passerelles IoT transmettent des mesures environnementales qui sont validées, analysées et transformées en alertes par plusieurs microservices.

ingestion-service constitue le point d'entrée de cette chaîne. Le service protège l'accès HTTP, contrôle les données reçues et publie un événement JSON dans Apache Kafka.

Vue d'ensemble UrbanHub

Diagramme Mermaid disponible dans le portail MkDocs et dans le dossier diagrammes.

Responsabilités du service
ingestion-service prend en charge :

1. l'exposition de la route REST d'ingestion ;
2. l'authentification de la passerelle par clé API ;
3. la limitation du débit avec un token bucket ;
4. la validation syntaxique et métier de la mesure ;
5. la génération de eventId et correlationId ;
6. la construction de MeasurementReceivedEvent ;
7. la publication JSON dans le topic measurements.received ;
8. l'exposition des endpoints de santé.

Le service ne calcule pas l'indice de qualité de l'air et ne produit pas de notification.
Ces responsabilités
appartiennent aux services en aval.

Architecture interne

Diagramme Mermaid disponible dans le portail MkDocs et dans le dossier diagrammes.

Découpage par couches

Couche API
Package :

com.urbanhub.ingestion.api

Responsabilités :

- exposer les routes HTTP ;
- désérialiser le JSON ;

- appliquer Bean Validation ;
- traduire la requête en commande applicative ;
- retourner une réponse HTTP structurée ;
- standardiser les erreurs.

Classes représentatives :

- IngestionController ;
- MeasurementIngestionRequest ;
- MeasurementIngestionResponse ;
- GlobalExceptionHandler ;
- ApiErrorResponse.

Couche application

Package :

com.urbanhub.ingestion.application

Responsabilités :

- orchestrer le cas d'utilisation d'ingestion ;
- appliquer les invariants métier défensifs ;
- générer les identifiants ;
- construire l'événement ;
- appeler le port de publication.

Classes et interfaces représentatives :

- MeasurementIngestionService ;
- RawMeasurementCommand ;
- MeasurementReceivedPublisher ;
- CorrelationIdGenerator ;
- IngestionResult.

Couche domaine

Package :

com.urbanhub.ingestion.domain

La couche domaine contient les concepts métier indépendants de Spring, HTTP et Kafka. Cette séparation réduit le couplage et facilite les tests unitaires.

Couche événementielle

Package :

com.urbanhub.ingestion.events

Le record MeasurementReceivedEvent représente le contrat JSON publié dans Kafka.

Le contrat contient notamment :

- un identifiant unique ;
- un type et une version d'événement ;
- un identifiant de corrélation ;
- une date d'occurrence ;
- une source ;
- les données de mesure.

Couche messaging
Package :

com.urbanhub.ingestion.messaging

L'adaptateur Kafka implémente le port applicatif MeasurementReceivedPublisher. Le service applicatif ne dépend donc pas directement de KafkaTemplate.

Couche sécurité
Package :

com.urbanhub.ingestion.security

Responsabilités :

- extraire et comparer la clé API ;
- créer le contexte d'authentification ;
- imposer une politique stateless ;
- appliquer le refus par défaut ;
- limiter le débit par identité authentifiée ;
- produire une réponse HTTP 401 ou 429 structurée.

Flux d'une mesure

Diagramme Mermaid disponible dans le portail MkDocs et dans le dossier diagrammes.

Décisions techniques

Architecture événementielle

Kafka découple l'ingestion des traitements en aval. L'API peut accepter une mesure sans attendre la validation qualité ou le calcul d'une alerte.

Port et adaptateur

MeasurementIngestionService dépend d'une interface de publication. Cette inversion de dépendance :

- limite le couplage à Kafka ;
- simplifie les tests avec un mock ;
- permet de remplacer l'adaptateur technique sans modifier le cas d'utilisation.

Contrat JSON indépendant des packages Java

Le serializer Kafka ne publie pas d'information de type Java dans les headers. Les services échangent un contrat JSON versionné plutôt qu'un nom de classe interne.

Ce choix évite le couplage entre :

com.urbanhub.ingestion.events.MeasurementReceivedEvent

et la classe locale du consommateur.

Clé Kafka

La publication utilise zoneId comme clé Kafka. Les événements d'une même zone sont ainsi orientés de manière cohérente vers une partition, ce qui facilite le maintien de l'ordre relatif par zone.

Validation à deux niveaux

Bean Validation protège la frontière HTTP. Une validation métier défensive reste présente dans la couche application afin de protéger le cas d'utilisation si une autre interface l'appelle ultérieurement.

Sécurité stateless

Le service n'utilise pas de session HTTP. Chaque requête apporte sa preuve d'authentification et le serveur applique une politique de refus par défaut.

Rate limiting en mémoire

Bucket4j limite les appels dans l'instance courante. Cette solution est simple et adaptée au déploiement mono-instance de démonstration. Une architecture multi-instance nécessiterait Redis ou une API Gateway.

Déploiement conteneurisé

L'image Docker utilise un build multi-stage :

1. Maven compile et package l'application ;
2. une image Java Runtime minimale exécute uniquement le JAR final.

Le runtime :

- utilise Java 25 ;
- exécute le processus avec l'utilisateur non-root urbanhub ;
- expose le port 8082 ;
- limite l'utilisation mémoire de la JVM selon les ressources du conteneur.

Frontières de confiance

Client vers API

La passerelle IoT est considérée comme non fiable avant authentification et validation.

Contrôles :

- clé API ;
- rate limiting ;
- limites de taille et de format ;
- réponses d'erreur contrôlées.

API vers Kafka

Le flux transporte des événements métier JSON. Le contrat est versionné avec eventVersion.

Évolution recommandée : TLS, SASL et ACL par microservice.

Opérateur vers Redpanda Console

L'interface d'administration est limitée à localhost dans l'environnement de démonstration.

Qualités architecturales

Maintenabilité

- responsabilités séparées ;
- dépendances dirigées vers des abstractions ;
- contrats documentés ;
- tests par couche ;
- règles de qualité automatisées.

Testabilité Le port de publication et le générateur de corrélation peuvent être mockés. Les tests HTTP utilisent MockMvc sans broker réel.

Évolutivité L'architecture événementielle permet d'ajouter un consommateur sans modifier l'API d'ingestion. Les événements disposent d'un champ de version.

Observabilité eventId identifie un événement unique. correlationId permet de suivre un traitement distribué. Actuator expose la santé de l'instance.

Limites et trajectoire d'évolution

Limites actuelles

- clé API partagée ;
- quotas conservés en mémoire ;
- Kafka local sans politique complète TLS/SASL/ACL ;
- observabilité limitée aux logs et healthchecks ;
- absence de DLQ dans le périmètre actuel.

Évolutions recommandées

1. utiliser une identité distincte par passerelle ;
2. externaliser le rate limiting ;
3. sécuriser Kafka avec TLS, SASL et ACL ;
4. ajouter retry, backoff et DLQ ;
5. centraliser logs, métriques et traces ;
6. persister et superviser les contrats événementiels.

Référence API

Informations générales

Propriété	Valeur
Service	ingestion-service
URL locale	http://localhost:8082
Format	JSON
Authentification	Clé API dans le header X-API-Key
Documentation interactive	/swagger-ui.html
Spécification OpenAPI	/v3/api-docs

Authentification La route d'ingestion exige le header suivant :

X-API-Key: <clé-configurée>

La clé est fournie au service avec la variable d'environnement :

INGESTION_API_KEY

Une clé absente ou invalide produit une réponse HTTP 401. Aucun secret réel ne doit être enregistré dans le dépôt ou dans la documentation.

Ingérer une mesure

Requête

POST /api/ingestion/measurements
Content-Type: application/json
X-API-Key: <clé-configurée>

Corps JSON

```
{
  "zoneId": "ZFE-1",
  "stationId": "AIR-STATION-042",
  "indicator": "N02",
  "value": 220.5,
  "timestamp": "2026-07-27T08:00:00Z"
}
```

Schéma d'entrée

Champ	Type	Obligatoire	Contraintes
Exemple			
Non vide, 50 caractères zoneId chaîne Oui ZFE-1 maximum, lettres, chiffres, _ ou -			
Non vide, 100 caractères stationId chaîne Oui AIR-STATION-042 maximum, lettres, chiffres, _ ou -			
Une valeur parmi indicator N02	chaîne	Oui	N02, PM10, PM25
Entre 0 et 5 000 value 220.5	nombre décimal	Oui	inclus
Date passée ou timestamp 00Z	2026-07-27T08:00: date ISO 8601	Oui	présente
Réponse acceptée Statut :			
202 Accepted			
Corps :			
{ "status": "ACCEPTED", "correlationId": "identifiant-généré" }			
Schéma de sortie			
Champ	Type	Description	
Statut fonctionnel de la prise en status	chaîne	charge	
Identifiant utilisé pour suivre la correlationId chaîne distribuée	chaîne	mesure dans la	
Exemples de requêtes			

curl

```
curl -i -X POST http://localhost:8082/api/ingestion/measurements -H "Content-Type: application/json" -H "X-API-Key: replace-with-a-local-key" -d '{
  "zoneId": "ZFE-1",
  "stationId": "AIR-STATION-042",
  "indicator": "N02",
  "value": 220.5,
  "timestamp": "2026-07-27T08:00:00Z"
}'
```

PowerShell

```
$body = @{
    zoneId = "ZFE-1"
    stationId = "AIR-STATION-042"
    indicator = "N02"
    value = 220.5
    timestamp = "2026-07-27T08:00:00Z"
} | ConvertTo-Json
```

```
Invoke-RestMethod `
-Method Post `
-Uri "http://localhost:8082/api/ingestion/measurements" `
-Headers @{
    "X-API-Key" = $env:INGESTION_API_KEY
} `
-ContentType "application/json" `
-Body $body
```

Codes de réponse

Statut Action recommandée	Code d'erreur	Signification
Conserver le 202 correlationId	Aucun	Mesure acceptée
Un ou plusieurs champs 400 indiqués	Corriger les champs VALIDATION_ERROR	sont invalides
Vérifier la syntaxe et le 400 Content-Type	MALFORMED_JSON	JSON absent ou illisible
Invariant métier non 400 Corriger la mesure	INVALID_MEASUREMENT	respecté
401 Fournir une clé valide	UNAUTHORIZED	Clé API absente ou invalide
Respecter le header 429 Retry-After	RATE_LIMIT_EXCEEDED	Quota de requêtes épuisé
Consulter les logs et le 500 guide de dépannage	Variable	Erreur interne non prévue
Format d'erreur standard <pre>{ "timestamp": "2026-07-29T10:00:00Z", "status": 400, "error": "VALIDATION_ERROR", "message": "The request contains invalid fields", "path": "/api/ingestion/measurements", "fieldErrors": { "indicator": "indicator must be one of: N02, PM10, PM25" } }</pre>		
Champs d'erreur		
Champ	Type	Description
timestamp création de la réponse	date ISO 8601	Date de
status	entier	Statut HTTP

Champ	Type	Description
error	chaîne	Code stable de l'erreur
message	chaîne	Résumé lisible
path	chaîne	Route appelée
fieldErrors champ	objet	Erreurs indexées par

Rate limiting

Le service applique un token bucket par identité authentifiée.

Headers de réponse

Header	Description
X-Rate-Limit-Remaining	Nombre de jetons encore disponibles
Délai minimal avant une nouvelle tentative après HTTP Retry-After	429

Exemple HTTP 429

```
{
  "timestamp": "2026-07-29T10:00:00Z",
  "status": 429,
  "error": "RATE_LIMIT_EXCEEDED",
  "message": "Too many ingestion requests",
  "path": "/api/ingestion/measurements",
  "fieldErrors": {}
}
```

Santé du service

Readiness

GET /actuator/health/readiness

Réponse attendue :

```
{
  "status": "UP"
}
```

Information

GET /actuator/info

Seuls les endpoints Actuator explicitement autorisés doivent être exposés.

Swagger et OpenAPI

Lorsque la documentation interactive est activée :

<http://localhost:8082/swagger-ui.html>

La spécification JSON générée est disponible à :

<http://localhost:8082/v3/api-docs>

Une copie exportée est fournie dans :

openapi.json

Événement produit

Une mesure acceptée produit un événement dans :

Topic : measurements.received
Clé Kafka : zoneId
Type : MeasurementReceived
Version : 1.0

Exemple :

```
{
  "eventId": "identifiant-généré",
  "eventType": "MeasurementReceived",
  "eventVersion": "1.0",
  "correlationId": "identifiant-de-corrélation",
  "occurredAt": "2026-07-29T10:00:00Z",
  "source": "ingestion-service",
  "zoneId": "ZFE-1",
  "stationId": "AIR-STATION-042",
  "indicator": "NO2",
  "value": 220.5,
  "timestamp": "2026-07-27T08:00:00Z"
}
```

Le contrat JSON ne dépend pas du nom complet d'une classe Java.

Pipeline CI/CD

Objectif Le pipeline automatise l'installation, les tests, la qualité, la sécurité, la création de l'image Docker et le déploiement local de ingestion-service.

Chaîne d'exécution

Diagramme Mermaid disponible dans le portail MkDocs et dans le dossier diagrammes.

Chaque job dépend du précédent. Une erreur interrompt la chaîne et protège la branche principale.

Déclencheurs Le workflow est exécuté :

- lors d'un push sur main affectant le service ou le workflow ;
- lors d'une pull request vers main ;
- manuellement avec workflow_dispatch.

Job install
Responsabilités :

- récupérer le dépôt ;
- installer Java 25 Temurin ;
- restaurer le cache Maven ;
- afficher les versions Java et Maven ;
- résoudre les dépendances avec dependency:go-offline.

Job test
Le job exécute :

```
mvn --batch-mode --no-transfer-progress clean verify
```

La commande couvre :

- compilation ;
- tests unitaires ;
- tests MockMvc ;
- tests de validation ;
- tests d'authentification ;
- test du rate limiting ;
- test non fonctionnel du temps de réponse ;
- rapport JaCoCo ;

- gates de couverture ;
- packaging du JAR.

Couverture

Couverture des lignes $\geq 60\%$ Couverture des branches $\geq 60\%$

Les rapports Surefire et JaCoCo sont publiés dans l'artefact ingestion-test-reports.

Job quality

Checkstyle Checkstyle contrôle les conventions Java et bloque le job en cas de violation.

SpotBugs et Find Security Bugs

SpotBugs analyse le bytecode. Find Security Bugs ajoute des détecteurs orientés sécurité.

Résultat initial :

1 finding général

0 finding SECURITY

SpotBugs reste temporairement en mode audit pendant la qualification du finding général. Les rapports sont publiés dans ingestion-quality-reports.

Job security

Gitleaks Gitleaks analyse l'historique Git complet avec les secrets masqués dans les sorties.

Secret détecté -> job en échec

Une exclusion n'est autorisée qu'après qualification d'un faux positif et doit rester limitée au fingerprint exact.

Trivy filesystem

Trivy analyse les dépendances Maven et produit :

- un rapport JSON ;
- un rapport SARIF ;
- une SBOM CycloneDX.

La baseline Trivy reste temporairement non bloquante pendant la qualification des vulnérabilités existantes. L'absence des rapports attendus bloque néanmoins le job.

Les résultats sont publiés dans ingestion-security-reports.

Job build

Le job construit une image Docker versionnée :

urbanhub-ingestion:<SHA-Git>

Propriétés :

- build multi-stage ;
- compilation Maven dans le premier stage ;
- runtime Java 25 dans le second stage ;
- copie du seul JAR nécessaire ;
- exécution avec l'utilisateur non-root urbanhub ;
- port applicatif 8082.

Les métadonnées de l'image sont publiées dans ingestion-build-evidence.

Job deploy

Le job réalise un déploiement local éphémère :

1. générer une clé API temporaire et masquée ;
2. démarrer Kafka et ingestion-service avec Docker Compose ;
3. attendre la readiness ;
4. vérifier qu'une requête sans clé retourne HTTP 401 ;
5. vérifier qu'une requête authentifiée retourne HTTP 202 ;
6. collecter l'état et les logs ;
7. arrêter et nettoyer l'environnement.

Les preuves sont publiées dans ingestion-deployment-evidence.

Artefacts

Artefact	Contenu
ingestion-test-reports	Surefire et JaCoCo
ingestion-quality-reports	Checkstyle et SpotBugs
ingestion-security-reports	Gitleaks, Trivy et SBOM
ingestion-build-evidence	Métadonnées de l'image Docker

ingestion-deployment-evidence Readiness, smoke tests et logs

Gates bloquantes

Le pipeline échoue en cas de :

- dépendance Maven non résolue ;
- erreur de compilation ;
- test en échec ;

- violation Checkstyle ;
- couverture insuffisante ;
- secret détecté ;
- image Docker non construite ;
- readiness indisponible ;
- smoke test en échec.

Reproductibilité locale

```
mvn --file services/ingestion-service/pom.xml clean verify
```

```
docker build --tag urbanhub-ingestion:local services/ingestion-service
```

```
export INGESTION_API_KEY="replace-with-a-local-key"
docker compose up --build -d kafka ingestion-service
```

Sous PowerShell :

```
$env:INGESTION_API_KEY = "replace-with-a-local-key"
docker compose up --build -d kafka ingestion-service
```

Résultat validé

Le pipeline complet a été exécuté avec succès de install à deploy. Les tests, les gates de couverture, les contrôles qualité, les analyses de sécurité, le build Docker et les smoke tests ont abouti.

Guide d exploitation

Démarrage Définir une clé API locale sous PowerShell :

```
$env:INGESTION_API_KEY = "replace-with-a-local-key"
```

Démarrer Kafka et le service :

```
docker compose up --build -d kafka ingestion-service
```

État des conteneurs
docker compose ps

Readiness
Invoke-RestMethod `
http://localhost:8082/actuator/health/readiness

Résultat attendu :

```
{  
  "status": "UP"  
}
```

Envoyer une mesure
\$body = @{
 zoneId = "ZFE-1"
 stationId = "AIR-STATION-042"
 indicator = "N02"
 value = 220.5
 timestamp = "2026-07-27T08:00:00Z"
} | ConvertTo-Json

Invoke-RestMethod `
-Method Post `
-Uri "http://localhost:8082/api/ingestion/measurements" `
-Headers @{ "X-API-Key" = \$env:INGESTION_API_KEY } `
-ContentType "application/json" `
-Body \$body

Logs Service d'ingestion :

```
docker compose logs -f ingestion-service
```

Kafka :

```
docker compose logs -f kafka
```

Vérifications après déploiement
Après chaque déploiement :

1. vérifier l'état des conteneurs ;
2. vérifier /actuator/health/readiness ;
3. tester une requête sans clé et attendre HTTP 401 ;
4. tester une requête valide et attendre HTTP 202 ;
5. vérifier le topic measurements.received ;
6. contrôler les logs d'erreur.

Reconstruction ciblée
`docker compose build --no-cache ingestion-service`
`docker compose up -d --force-recreate ingestion-service`

Redémarrage
`docker compose restart ingestion-service`

Arrêt normal
`docker compose down`

Nettoyage complet
`docker compose down --volumes --remove-orphans`

Sauvegarde et persistance
ingestion-service ne possède pas de base de données dans le périmètre actuel. Kafka gère les événements selon sa configuration de rétention. Le rate limiting est conservé en mémoire et réinitialisé au redémarrage du service.

Supervision minimale
À surveiller :

- statut de readiness ;
- redémarrages du conteneur ;
- erreurs de publication Kafka ;
- taux de réponses HTTP 400, 401, 429 et 500 ;
- temps de réponse de l'API ;
- espace disque et disponibilité du broker ;
- lag des consommateurs en aval.

Rotation de la clé API

Procédure :

1. générer une nouvelle valeur forte ;
2. mettre à jour la source de secret de l'environnement ;
3. recréer le conteneur ;
4. transmettre la nouvelle clé aux passerelles autorisées ;
5. vérifier HTTP 202 avec la nouvelle clé ;
6. vérifier HTTP 401 avec l'ancienne clé ;
7. ne jamais enregistrer la clé dans Git ou dans les logs.

Dépannage

Le service ne démarre pas

Message lié à INGESTION_API_KEY
Cause : la clé obligatoire n'est pas définie.

```
$env:INGESTION_API_KEY = "replace-with-a-local-key"
docker compose up -d --force-recreate ingestion-service
```

HTTP 401 Unauthorized
Causes possibles :

- header X-API-Key absent ;
- clé incorrecte ;
- conteneur créé avec une ancienne valeur.

```
docker compose config
docker compose logs ingestion-service
docker compose up -d --force-recreate ingestion-service
```

HTTP 400 Bad Request
Vérifier :

```
zoneId non vide
stationId non vide
indicator parmi N02, PM10, PM25
value entre 0 et 5000
timestamp non futur
Content-Type application/json
```

HTTP 429 Too Many Requests
Le quota est épuisé.

- attendre le délai Retry-After ;
- vérifier l'absence de boucle côté client ;
- ajuster la configuration uniquement après validation du besoin.

Kafka indisponible
Symptômes possibles :

```
Bootstrap broker disconnected
Connection to node could not be established
```

Diagnostic :

```
docker compose ps
docker compose logs kafka
docker compose logs ingestion-service
```

Résolution :

```
docker compose restart kafka
docker compose restart ingestion-service
```

Dans Docker, l'adresse attendue est généralement kafka:29092 selon la configuration Compose.

```
Healthcheck unhealthy
docker inspect ingestion-service `
  --format "{{json .State.Health}}"
```

```
docker compose logs ingestion-service
```

Causes possibles :

- démarrage incomplet ;
- port incorrect ;
- endpoint Actuator non exposé ;
- erreur de configuration ;
- dépendance Kafka indisponible.

Checkstyle ne trouve pas sa configuration
Message :

Unable to find configuration file: config/checkstyle/checkstyle.xml

Le fichier attendu est :

services/ingestion-service/config/checkstyle/checkstyle.xml

Le pom.xml doit utiliser :

```
${project.basedir}/config/checkstyle/checkstyle.xml
```

Le Dockerfile doit copier explicitement le fichier avant le build Maven.

Échec de la couverture JaCoCo
Message possible :

Coverage checks have not been met

Procédure :

1. ouvrir target/site/jacoco/index.html ;
2. identifier les classes et branches non couvertes ;
3. ajouter des tests utiles ;
4. ne pas réduire le seuil uniquement pour rendre le build vert.

SpotBugs remonte un finding

- identifier le type, la classe et la ligne ;
- qualifier le vrai ou faux positif ;
- corriger le code si nécessaire ;
- documenter toute acceptation temporaire ;
- ne pas ajouter une exclusion globale sans justification.

Gitleaks bloque le pipeline

Pour un vrai secret :

1. révoquer le secret ;
2. créer une nouvelle valeur ;
3. retirer le secret du code ;
4. purger l'historique si nécessaire ;
5. relancer le scan.

Pour un faux positif :

1. prouver que la valeur n'est pas opérationnelle ;
2. retirer les rapports générés du dépôt ;
3. documenter la qualification ;
4. limiter l'exclusion au fingerprint exact.

Trivy reçoit HTTP 429 de Maven Central

Le runner partagé est limité temporairement.

Le pipeline doit :

1. restaurer le cache Maven ;
2. exécuter dependency:go-offline ;
3. monter ~/.m2 en lecture seule dans Trivy.

Build Docker en échec

```
docker build `
  --no-cache `
  --progress=plain `
  -t urbanhub-ingestion:diagnostic `
  services/ingestion-service
```

Vérifier :

- pom.xml ;
- src/ ;
- la configuration Checkstyle ;
- .dockerignore ;
- l'accès à Maven Central.

Réinitialisation locale

```
docker compose down --volumes --remove-orphans  
docker compose up --build -d kafka ingestion-service
```

docker system prune ne doit être utilisé qu'en connaissance de son impact sur les ressources Docker inutilisées.

Maintenance et évolution

Objectif Ce guide explique comment reprendre, maintenir et faire évoluer le microservice ingestion-service sans dégrader son contrat API, ses garanties de sécurité ou son pipeline CI/CD.

Périmètre du service
ingestion-service est responsable de :

1. recevoir les mesures IoT par API REST ;
2. authentifier les passerelles avec une clé API ;
3. limiter la fréquence des requêtes ;
4. valider les données entrantes ;
5. générer les identifiants techniques ;
6. publier un événement MeasurementReceived dans Apache Kafka.

Le service ne calcule pas la qualité de l'air et ne prépare pas les notifications. Ces responsabilités appartiennent aux microservices situés en aval.

Reprendre le projet

1. Vérifier les prérequis

```
java --version
mvn --version
docker --version
docker compose version
```

Versions de référence :

- Java 25 ;
- Maven 3.9 ou version compatible ;
- Docker avec Compose V2.

2. Installer les dépendances Depuis la racine du dépôt :

```
mvn --file services/ingestion-service/pom.xml --batch-mode --no-transfer-progress
dependency:go-offline
```

3. Exécuter les contrôles locaux

```
mvn --file services/ingestion-service/pom.xml clean verify
```

La commande doit valider :

- Checkstyle ;

- les tests JUnit ;
- le test non fonctionnel ;
- la couverture JaCoCo ;
- les seuils de couverture ;
- SpotBugs et Find Security Bugs ;
- le packaging du JAR.

4. Construire l'image Docker

```
docker build --tag urbanhub-ingestion:local services/ingestion-service
```

5. Démarrer l'environnement local Définir une clé API sans la versionner :

```
$env:INGESTION_API_KEY = "replace-with-a-local-key"
```

Puis :

```
docker compose up --build -d kafka ingestion-service
```

Organisation du code

```
src/main/java/com/urbanhub/ingestion/
├── api/           Contrôleurs, DTO REST et gestion des erreurs
├── application/   Cas d'utilisation, commandes et ports
├── config/        Configuration Kafka et OpenAPI
├── domain/        Modèle métier indépendant de l'infrastructure
├── events/        Contrats événementiels publiés
├── messaging/     Adaptateurs Kafka
└── security/      Authentification et rate limiting
```

Règles de dépendance

- la couche domain ne dépend pas de Spring ou de Kafka ;
- la couche application dépend de ports, pas directement de KafkaTemplate ;
- la couche api traduit HTTP vers les commandes applicatives ;
- la couche messaging implémente les ports de publication ;
- les données invalides sont rejetées avant toute publication Kafka.

Faire évoluer l'API REST

Pour ajouter ou modifier un champ :

1. modifier MeasurementIngestionRequest ;
2. ajouter les contraintes Bean Validation nécessaires ;
3. adapter RawMeasurementCommand ;
4. adapter le mapping du contrôleur ;
5. ajouter ou mettre à jour les tests MockMvc ;
6. vérifier les réponses d'erreur ;
7. régénérer openapi.json ;

8. mettre à jour docs/api.md et CHANGELOG.md.

Compatibilité Une suppression, un renommage ou un changement de type constitue potentiellement une rupture de contrat. Dans ce cas :

- introduire une nouvelle version d'API ;
- documenter la période de transition ;
- conserver temporairement l'ancien contrat si nécessaire ;
- ajouter des tests de compatibilité.

Faire évoluer les événements Kafka
L'événement `MeasurementReceived` contient un champ `eventVersion`.

Pour faire évoluer le contrat :

1. privilégier l'ajout de champs optionnels ;
2. éviter de renommer ou supprimer directement un champ ;
3. incrémenter `eventVersion` pour une évolution incompatible ;
4. adapter les consommateurs avant le producteur lorsque cela est nécessaire ;
5. conserver `eventId` et `correlationId` ;
6. mettre à jour la documentation du contrat ;
7. tester la désérialisation côté consommateur.

Les noms complets des classes Java ne doivent pas devenir des contrats interservices. Les producteurs publient du JSON sans dépendance aux packages internes.

Faire évoluer la sécurité

Clé API

La clé API actuelle est adaptée au périmètre pédagogique et à une instance locale. Pour une production multi-capteurs :

- attribuer une identité distincte à chaque passerelle ;
- stocker les secrets dans un gestionnaire dédié ;
- prévoir la rotation et la révocation ;
- envisager une signature HMAC, OAuth2 client credentials ou mTLS.

Rate limiting

Le rate limiting est stocké en mémoire. Pour plusieurs répliquas :

- externaliser les buckets dans Redis ou un stockage partagé ;
- ou centraliser la limitation dans une API Gateway ;
- conserver les réponses HTTP 429 et le header `Retry-After`.

Kafka

Évolutions recommandées :

- activer TLS ;
- utiliser SASL pour authentifier les services ;
- définir des ACL par topic ;
- limiter les droits de chaque microservice au strict nécessaire.

Faire évoluer les tests

Toute évolution fonctionnelle doit inclure :

- un test nominal ;
- au moins un test d'erreur ;
- un test de non-régression lorsque la modification corrige un défaut ;
- une mise à jour des tests de contrat si l'API ou un événement change.

Les seuils JaCoCo sont :

Couverture des lignes ≥ 60 % Couverture des branches ≥ 60 %

Une baisse sous ces seuils bloque le build.

Maintenir les dépendances

Procédure recommandée :

1. consulter régulièrement les alertes de dépendances ;
2. lancer Trivy sur le service et sur l'image finale ;
3. identifier la dépendance directe ou transitive ;
4. vérifier la version corrigée ;
5. mettre à jour une dépendance à la fois ;
6. exécuter `mvn clean verify` ;
7. reconstruire et rescanner l'image ;
8. mettre à jour la SBOM CycloneDX ;
9. documenter la modification dans le changelog.

Une vulnérabilité ne doit pas être masquée sans justification, propriétaire et date de révision.

Maintenir le pipeline CI/CD

Le pipeline suit l'ordre :

`install -> test -> quality -> security -> build -> deploy`

Lors d'une évolution du pipeline :

- conserver les dépendances entre jobs ;
- maintenir les permissions au minimum ;
- ne pas ajouter `continue-on-error` aux gates bloquantes ;
- pinner les versions des outils ;

- vérifier que les artefacts restent produits ;
- tester un run complet avant fusion ;
- mettre à jour docs/pipeline.md.

Journal des versions

Chaque évolution significative doit être ajoutée à CHANGELOG.md dans une catégorie adaptée :

Added Changed Fixed Security Deprecated Removed

Les commits suivent Conventional Commits, par exemple :

feat(security): add ingestion rate limiting fix(ci): provide Maven cache to Trivy test: add ingestion response time test docs: document pipeline troubleshooting

Définition de terminé

Une évolution est terminée lorsque :

- le code respecte les responsabilités architecturales ;
- les tests passent ;
- Checkstyle passe ;
- les gates JaCoCo passent ;
- les analyses de sécurité sont exécutées ;
- l'image Docker est construite ;
- les smoke tests passent ;
- OpenAPI est à jour si le contrat change ;
- la documentation et le changelog sont à jour.

Dette technique et feuille de route

Priorité élevée

- qualifier et corriger les vulnérabilités Trivy ;
- terminer la qualification du finding SpotBugs ;
- mettre en place TLS et des ACL Kafka ;
- remplacer la clé partagée par des identités distinctes.

Priorité moyenne

- externaliser le rate limiting ;
- ajouter une DLQ et une stratégie de retry ;
- centraliser les logs et les métriques ;

- automatiser la mise à jour des dépendances.

Priorité basse

- versionner plusieurs versions du portail avec mike ;
- publier automatiquement les release notes ;
- ajouter des tests de charge distribués.

Sécurité

Objectifs Les contrôles protègent :

- l'authenticité des passerelles ;
- l'intégrité des mesures ;
- la disponibilité de l'API ;
- la confidentialité des secrets ;
- la supply chain logicielle ;
- l'exécution du conteneur.

Authentification par clé API

La route d'ingestion exige X-API-Key. La clé attendue provient de INGESTION_API_KEY et n'est pas stockée en clair dans le dépôt.

La comparaison utilise une méthode adaptée aux données sensibles. Le service reste stateless et ne crée pas de session HTTP.

Limite : une clé partagée ne fournit pas une identité distincte par capteur. Une production mature utilisera HMAC, OAuth2 client credentials ou mTLS.

Autorisation La configuration applique le refus par défaut. Seules les routes explicitement autorisées restent accessibles.

Les endpoints de santé et la documentation interactive sont ouverts uniquement selon la configuration de l'environnement.

Validation des entrées

Bean Validation contrôle :

- présence des champs ;
- longueur des identifiants ;
- caractères autorisés ;
- liste des indicateurs ;
- plage de valeur ;
- cohérence temporelle.

Une validation métier défensive protège aussi le cas d'utilisation en dehors de l'interface HTTP.

Rate limiting

Bucket4j applique un token bucket par identité authentifiée. Un quota épuisé retourne HTTP 429 et Retry-After.

Limite : les buckets sont en mémoire. Une architecture multi-instance doit employer Redis ou une API Gateway.

Gestion des erreurs

Les réponses n'exposent ni stack trace, ni configuration Kafka, ni secret. Les erreurs utilisent un format JSON homogène avec un code stable.

Sécurité Kafka

État actuel : environnement local de démonstration.

Évolutions recommandées :

- TLS ;
- authentification SASL ;
- ACL par microservice et topic ;
- quotas producteurs ;
- stratégie retry et DLQ ;
- supervision du lag.

Sécurité du conteneur

L'image :

- utilise un build multi-stage ;
- contient uniquement le runtime final ;
- s'exécute avec l'utilisateur non-root urbanhub ;
- utilise un tag immuable fondé sur le SHA Git dans le pipeline.

Évolutions recommandées :

- filesystem read-only ;
- no-new-privileges ;
- suppression des capacités Linux ;
- scan périodique des images ;
- signature de l'image.

Gitleaks Gitleaks analyse l'historique complet et bloque le pipeline lorsqu'un secret est détecté.

Un faux positif doit être qualifié et exclu uniquement par fingerprint exact. Un vrai secret doit être révoqué avant toute autre action.

Trivy et dépendances

Trivy analyse les dépendances Maven et génère des rapports JSON et SARIF.

La baseline existante doit être priorisée selon :

- sévérité ;
- disponibilité d'un correctif ;
- exploitabilité ;
- criticité du composant ;
- exposition réelle.

La gate Trivy reste temporairement en audit pendant la qualification initiale. La cible est de bloquer les vulnérabilités critiques corrigibles.

SpotBugs et Find Security Bugs

SpotBugs analyse le bytecode. Find Security Bugs ajoute des règles de sécurité Java.

Résultat initial : zéro finding classé SECURITY. Ce résultat ne remplace pas les autres contrôles, car un SAST ne couvre ni les secrets, ni les CVE des dépendances, ni la configuration runtime.

SBOM CycloneDX

La SBOM inventorie les composants logiciels. Elle permet de répondre rapidement à une nouvelle CVE et améliore la traçabilité de la supply chain.

Threat model synthétique

Risque	Contrôle actuel	Risque résiduel
Usurpation de capteur	Clé API	Clé partagée
Flood HTTP instance	Rate limiting	Quota local à une
Payload malformé à maintenir	Validation stricte	Cas métier futurs
Secret commité qualifier	Gitleaks bloquant	Faux positifs à
Dépendance vulnérable	Trivy et SBOM	Baseline à remédier
Privilèges conteneur complémentaire possible	Utilisateur non-root	Hardening

Kafka compromis Réseau local restreint TLS, SASL et ACL à ajouter

Réponse à incident

En cas de fuite de secret :

1. révoquer immédiatement ;
2. identifier l'étendue ;
3. remplacer dans les environnements ;
4. retirer du dépôt et de l'historique ;
5. vérifier les journaux d'accès ;

6. relancer Gitleaks ;
7. documenter l'incident.

En cas de CVE critique :

1. identifier les composants avec la SBOM ;
2. vérifier l'exposition ;
3. mettre à jour ou mitiger ;
4. exécuter les tests ;
5. reconstruire l'image ;
6. rescanner ;
7. documenter la remédiation.

Référence technique

Versions

Composant	Version de référence
Java	25
Spring Boot	3.5.16 pour ingestion-service
Maven	3.9 ou compatible
JaCoCo	0.8.15
Maven Checkstyle Plugin	3.6.0
Checkstyle	13.9.0
SpotBugs Maven Plugin	4.10.3.0
Find Security Bugs	1.14.0
Gitleaks	8.30.1
Trivy	0.72.0

Ports

Composant	Port local
ingestion-service	8082
Kafka externe	9092
Redpanda Console	8090
Les ports administratifs sont limités à localhost dans l'environnement local.	

Variables d'environnement

Variable	Défaut	Description
INGESTION_API_KEY	Aucun en production	Clé API
SERVER_PORT	8082	Port HTTP
KAFKA_BOOTSTRAP_SERVERS	localhost:9092 hors Docker	Brokers Kafka
OPENAPI_ENABLED	Selon profil	Active OpenAPI
SWAGGER_ENABLED	Selon profil	Active Swagger UI
INGESTION_RATE_LIMIT_CAPACITY	100	Capacité du bucket
INGESTION_RATE_LIMIT_REFILL_TOK ENS	100	Jetons rechargés

Variable	Défaut	Description
INGESTION_RATE_LIMIT_DURATION_S recharge ECONDS	60	Période de

Routes

Méthode Résultat nominal	Route	Authentification
/api/ingestion/measur POST HTTP 202	ents	X-API-Key
/actuator/health/readin GET HTTP 200	Publique selon ess	configuration
Publique selon GET HTTP 200	/actuator/info	configuration
GET OpenAPI JSON	/v3/api-docs	Démonstration
GET Interface Swagger	/swagger-ui.html	Démonstration

Topics Kafka

Topic principal	Producteur	Consommateur
measurements.received service	ingestion-service	quality-
measurements.validated service	quality-service	air-quality-
measurements.rejected	quality-service	Supervision
air-quality.alert.detected service	air-quality-service	alerting-

Événement MeasurementReceived

Champ	Type	Description
eventId unique	chaîne	Identifiant
eventType MeasurementReceived	chaîne	
eventVersion contrat	chaîne	Version du
correlationId distribuée	chaîne	Corrélation
occurredAt publication	instant	Date de
source service	chaîne	ingestion-
zoneId	chaîne	Zone urbaine
stationId émettrice	chaîne	Station

indicator chaîne NO2, PM10 ou PM25

value	nombre	Valeur mesurée
-------	--------	----------------

Champ	Type	Description
timestamp	instant	Date de mesure
Commandes Maven <pre>mvn --file services/ingestion-service/pom.xml dependency:go-offline mvn --file services/ingestion-service/pom.xml clean test mvn --file services/ingestion-service/pom.xml clean verify mvn --file services/ingestion-service/pom.xml checkstyle:check mvn --file services/ingestion-service/pom.xml spotbugs:spotbugs</pre>		
Commandes Docker <pre>docker build -t urbanhub-ingestion:local services/ingestion-service docker compose up --build -d kafka ingestion-service docker compose ps docker compose logs -f ingestion-service docker compose down --volumes --remove-orphans</pre>		

Rapports générés

Rapport	Chemin Maven ou artefact
Surefire	target/surefire-reports/
JaCoCo HTML	target/site/jacoco/index.html
JaCoCo XML	target/site/jacoco/jacoco.xml
Checkstyle	target/checkstyle-result.xml
SpotBugs	target/spotbugsXml.xml
Trivy	Artefact ingestion-security-reports
SBOM	urbanhub-ingestion-sbom.cdx.json
Niveaux de qualité Lignes couvertes >= 60 % Branches couvertes >= 60 % Checkstyle : 0 violation Tests : 0 échec et 0 erreur Gitleaks : 0 secret non qualifié	

Codes HTTP

Code	Signification
202	Mesure acceptée
400	Requête invalide
401	Authentification invalide

Code	Signification
429	Quota dépassé
500	Erreur interne

Journal des versions

Toutes les modifications notables d'UrbanHub Ingestion Service sont documentées dans ce fichier.

Le format s'inspire de Keep a Changelog et les commits suivent Conventional Commits.

[Unreleased]

Added

- Documentation technique structurée selon Diátaxis.
- Portail MkDocs Material.
- Diagrammes Mermaid d'architecture, de classes et de séquence.
- Référence OpenAPI exportée.

Changed

- Aucun changement fonctionnel non publié.

[1.1.0] - 2026-07-29

Added

- Authentification de l'API d'ingestion avec X-API-Key.
- Rate limiting Bucket4j avec HTTP 429 et Retry-After.
- Validation stricte des requêtes avec Bean Validation.
- Réponses d'erreur JSON standardisées.
- Test non fonctionnel du temps de réponse.
- Gates JaCoCo à 60 % pour les lignes et les branches.
- Checkstyle bloquant.
- SpotBugs et Find Security Bugs.
- Gitleaks bloquant sur l'historique Git.
- Trivy filesystem, rapports JSON et SARIF.
- SBOM CycloneDX.
- Pipeline EC03 complet : install, test, quality, security, build et deploy.
- Smoke tests de readiness, HTTP 401 et HTTP 202.

Changed

- Image Docker multi-stage sous Java 25.
- Exécution du conteneur avec l'utilisateur non-root urbanhub.
- Interfaces Kafka administratives limitées à localhost.
- Configuration Checkstyle référencée par \${project.basedir}.

Fixed

- Copie de la configuration Checkstyle dans le stage de build Docker.

- Utilisation du cache Maven par Trivy dans GitHub Actions.
- Gestion d'un résultat vide dans la synthèse de sécurité.
- Qualification d'un faux positif Gitleaks provenant d'un rapport de test généré.

Security

- Refus des requêtes non authentifiées.
- Limitation des requêtes excessives.
- Détection des secrets, vulnérabilités de dépendances et défauts SAST.
- Publication d'une nomenclature logicielle CycloneDX.

[1.0.0] - 2026-05-06

Added

- API REST d'ingestion des mesures IoT.
- Publication de MeasurementReceived dans Kafka.
- Identifiants eventId et correlationId.
- Contrats événementiels JSON versionnés.
- Tests unitaires et tests MockMvc.
- Docker Compose avec Kafka et Redpanda Console.
- Documentation initiale des contrats et de l'architecture.

Déclaration d'utilisation de l'intelligence artificielle
Déclaration d'utilisation de l'intelligence artificielle

Responsabilité et principe d'utilisation
L'intelligence artificielle a été utilisée comme outil d'assistance à l'analyse, à la rédaction, au débogage et à la structuration. Les réponses produites n'ont pas été intégrées automatiquement. Chaque proposition a été relue, comparée au code réel, adaptée à l'environnement du projet, puis validée par compilation, tests automatisés, scans de sécurité ou exécution Docker.

L'architecture, la sécurité, la qualité du code et la décision finale d'intégration restent sous la responsabilité de l'auteur du livrable.

Outil utilisé

Élément	Valeur
Outil	M365 Copilot
Modèle	Modèle fondé sur GPT-5 reasoning
Plateforme	Interface Web

Analyse, génération assistée, revue, débogage et Modes d'utilisation documentation

Périmètre d'utilisation
L'outil a été utilisé pour :

- analyser les exigences des parties 1 et 2 de l'épreuve ;
- proposer une organisation progressive du pipeline GitHub Actions ;
- assister le débogage de Maven, Kafka, Docker, GitHub Actions, Gitleaks et Trivy ;
- proposer des tests de validation, d'authentification, de rate limiting et de temps de réponse ;
- assister l'intégration de JaCoCo, Checkstyle, SpotBugs, Find Security Bugs, Gitleaks et Trivy ;
- structurer la documentation selon Diátaxis ;
- préparer les pages MkDocs, les diagrammes Mermaid, la référence API et le rapport client BLUF ;
- préparer le document PDF final à partir des contenus Markdown validés.

L'outil n'a pas été autorisé à décider seul des seuils de sécurité, à masquer automatiquement les findings ou à modifier les secrets et environnements distants.

Démarche de Context Engineering
Les demandes fournissaient systématiquement le contexte technique utile, notamment :

- Java 25 et Spring Boot 3.5.16 ;
- la structure réelle du service et les classes concernées ;
- les fragments du pom.xml, du Dockerfile et du workflow ;

- les sorties exactes de Maven et GitHub Actions ;
- les erreurs observées, par exemple HTTP 429 de Maven Central ou fichier Checkstyle absent du contexte Docker ;
- les exigences du sujet d'examen et les critères C16 à C20 ;
- les résultats réels des tests, de JaCoCo, de SpotBugs, de Gitleaks et de Trivy ;
- les contraintes d'anonymat, de chemins relatifs et de reproductibilité.

Cette démarche a limité les réponses génériques et permis de comparer chaque proposition à l'état réel du projet.

Prompts majeurs et décisions associées

1. Sécurisation de l'API d'ingestion

Demande formulée : proposer une authentification par clé API, une validation stricte des payloads et un rate limiting compatible avec le service Spring Boot existant.

Réponse audité : la proposition a été validée après vérification de la chaîne de filtres Spring Security, du fonctionnement stateless, des statuts HTTP 401 et 429, de l'absence d'appel métier après dépassement de quota et de la non-publication Kafka pour une requête invalide.

Décision : intégration de la clé X-API-Key, de Bean Validation et de Bucket4j. Le choix d'une clé partagée a été accepté uniquement pour le périmètre pédagogique. Une identité par passerelle, HMAC, OAuth2 client credentials ou mTLS reste recommandée pour la production.

2. Découplage des événements Kafka

Demande formulée : supprimer le couplage entre les noms de packages Java du producteur et les consommateurs Kafka.

Réponse audité : une première proposition de serializer a été invalidée car elle ne correspondait pas à la version réelle de Spring Kafka. Le pom.xml et les propriétés du serializer ont été vérifiés avant adaptation.

Décision : conservation d'un contrat JSON explicite et versionné, sans information de type Java dans les headers. Cette solution a été retenue car elle réduit le couplage interservices et facilite l'évolution indépendante des consommateurs.

3. Gates de tests et de couverture

Demande formulée : ajouter une gate JaCoCo à 60 % et un test non fonctionnel de temps de réponse.

Réponse audité : les seuils proposés ont été comparés aux résultats réels, soit environ 78 % de couverture des instructions et 69 % des branches. Le test de performance a été exécuté avec une requête de chauffe afin de ne pas mesurer l'initialisation paresseuse de Spring.

Décision : intégration des gates de lignes et de branches à 60 %. Le test mesure la couche HTTP mais pas la latence Kafka, limite explicitement documentée.

4. Analyse de qualité Java

Demande formulée : intégrer Checkstyle, SpotBugs et Find Security Bugs sans masquer les findings existants.

Réponse audité : Checkstyle a d'abord été exécuté localement en mode bloquant. SpotBugs a détecté un finding général et aucun finding de catégorie SECURITY.

Décision : Checkstyle reste bloquant. SpotBugs reste temporairement en audit tant que le finding général n'est pas complètement qualifié. Aucune suppression globale n'a été ajoutée.

5. Gestion du finding Gitleaks

Demande formulée : traiter une détection generic-api-key dans un rapport XML de test commité dans l'historique.

Réponse audité : le chemin, le commit, la règle et le contenu ont été examinés. La valeur était fictive et limitée aux tests, mais le rapport généré n'avait pas à être versionné.

Décision : retrait du rapport généré, ajout au .gitignore, documentation du faux positif et exclusion limitée au fingerprint historique exact. La solution consistant à désactiver Gitleaks ou à utiliser continue-on-error a été rejetée, car elle aurait supprimé la gate de sécurité.

6. Incident Trivy et Maven Central

Demande formulée : corriger l'échec Trivy causé par HTTP 429 sur Maven Central.

Réponse audité : l'erreur indiquait explicitement un rate limiting externe et recommandait de préremplir le cache Maven.

Décision : restauration du cache avec actions/setup-java, exécution de dependency:go-offline, puis montage de ~/.m2 en lecture seule dans le conteneur Trivy. Un retry long a été rejeté, car le serveur imposait un délai de 1 800 secondes incompatible avec l'objectif de feedback rapide.

7. Build Docker

Demande formulée : corriger le build de l'image Java 25 et exécuter le service avec un utilisateur non-root.

Réponse audité : une proposition initiale n'a pas résolu le build. Les logs détaillés ont ensuite montré que Checkstyle ne trouvait pas config/checkstyle/checkstyle.xml dans le contexte Docker.

Décision : utilisation de \${project.basedir} dans Maven, copie explicite de la configuration Checkstyle, validation de sa présence pendant le build, runtime Java 25 Alpine, utilisateur urbanhub et port 8082. Le correctif a été validé par le job build puis par le déploiement et les smoke tests.

8. Documentation Doc as Code

Demande formulée : produire une documentation Partie 2 conforme à C20, structurée selon Diátaxis et destinée à MkDocs Material.

Réponse audité : chaque page a été rapprochée d'un objectif lecteur : tutoriel pour apprendre, guides pour agir, référence pour consulter et explications pour comprendre. Les commandes ont été formulées avec des chemins relatifs et les limites connues ont été conservées.

Décision : génération d'un README, de pages MkDocs, de diagrammes Mermaid, d'une référence API, d'un changelog, d'un rapport client BLUF et d'un PDF consolidé. La documentation ne présente pas l'application comme prête pour la production et indique les améliorations nécessaires.

Contrôles réalisés avant validation
Les vérifications suivantes ont été exécutées :

- mvn clean verify sous Java 25 ;
- tests JUnit et MockMvc sans échec ;
- gates JaCoCo satisfaites ;

- Checkstyle sans violation ;
- SpotBugs et Find Security Bugs exécutés ;
- Gitleaks exécuté sur l'historique complet ;
- Trivy et SBOM CycloneDX générés ;
- image Docker multi-stage construite ;
- utilisateur de runtime non-root vérifié ;
- readiness disponible ;
- requête sans clé rejetée avec HTTP 401 ;
- requête authentifiée acceptée avec HTTP 202 ;
- mkdocs build --strict exécuté avec succès.

Propositions invalidées ou adaptées

Les propositions suivantes n'ont pas été intégrées telles quelles :

- serializer Kafka incompatible avec la version du projet ;
- désactivation globale de Gitleaks, rejetée au profit d'une exclusion exacte et documentée ;
- simple retry de 30 minutes pour Trivy, remplacé par le cache Maven ;
- Dockerfile ne copiant pas la configuration Checkstyle ;
- comptage des vulnérabilités avec grep, fragile avec zéro résultat, remplacé par jq ;
- toute formulation laissant entendre que zéro finding SAST garantit l'absence de vulnérabilité.

Limites de l'assistance IA

- une réponse techniquement plausible peut être incompatible avec une version de dépendance ;
- une commande peut être valide sur Linux mais incorrecte sous PowerShell ou Git Bash ;
- les outils de sécurité peuvent produire des faux positifs ou ne pas couvrir toutes les vulnérabilités ;
- les choix de production, notamment les secrets, Kafka et l'observabilité, exigent une analyse humaine et contextuelle.

Responsabilité finale

Les décisions ont été retenues uniquement après vérification dans l'environnement réel.

L'auteur

assume la compréhension du fonctionnement, des limites, des risques résiduels et des évolutions recommandées. Toute nouvelle modification devra conserver la même démarche : contexte précis, revue critique, test reproductible et documentation du pourquoi.

Conclusion La solution fournit une base industrialisée, reproductible et documentée pour l'ingestion de mesures IoT. Le passage à la production doit prioriser des identités distinctes pour les passerelles, TLS/SASL/ACL pour Kafka, un rate limiting distribué, la remédiation des vulnérabilités et une observabilité centralisée.